PRAKTIKUM 6
Notifikasi Telegram Kondisi Sensor Yang dikirim oleh NodeMCU

NRP	: 1. 3223600046
	2. 3223600047
	3. 3223600056
	4. 3223600058
	5. 3223600060
Nama	: 1. Hari Tammim Mu'izha
	2. Bintang Endra Erlangga
	3. Mochamad Najib Thariq
	4. Ihsanta Zaki Sanjaya
	5. Mochammad Ega Yudhistira
Kelas	: 2 Teknik Komputer B
Mata Kuliah	: Praktikum Alat Pengembangan Perangkat Lunak
Tanggal	: Kamis, 27 Maret 2025
Dosen	: Dr. Setiawardhana, S.T., M.T.

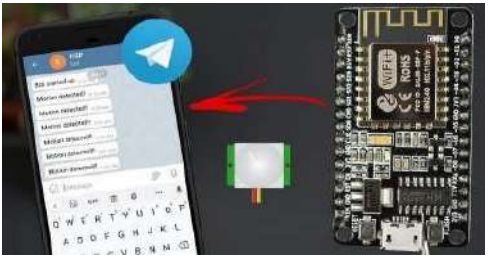
D4 TEKNIK KOMPUTER POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

IOT : NOTIFIKASI TELEGRAM KONDISI SENSOR YANG DIKIRIM OLEH NODEMCU

A. Deskripsi

Tutorial ini menunjukkan cara mengirim notifikasi ke akun Telegram Anda ketika ESP826 NodeMCU mendeteksi gerakan menggunakan sensor PIR. Selama Anda memiliki akses internet di ponsel, Anda akan mendapat pemberitahuan kondisi hasil deteksi gerakan. Pemrograman menggunakan Arduino IDE.



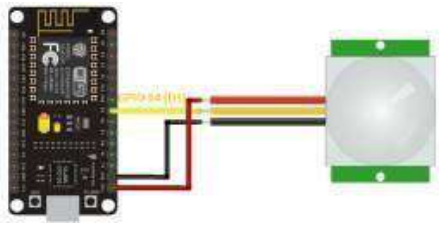
B. Project Overview

Diagram proses pembacaan sensor hingga pengiriman notifikasi oleh NodeMCU ke Telegram seperti ditunjukkan pada gambar 2.

C. Hardware & Software Required

1. NodeMCU ESP32 /ESP8266
2. PIR Sensor
3. Akun Telegram
4. Arduino IDE
5. Bread board
6. Jumper Wires

D. Schematic Diagram



E. Program Arduino IDE

```
/*

  Rui Santos

  Complete project details at https://RandomNerdTutorials.com/telegram-
esp8266-nodemcu-motion-detection-arduino/



  Project created using Brian Lough's Universal Telegram Bot Library:

  https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot

*/



#include  <ESP8266WiFi.h>

#include <WiFiClientSecure.h>

#include <UniversalTelegramBot.h>

#include <ArduinoJson.h>



// Replace with your network credentials

const char* ssid = "REPLACE_WITH_YOUR_SSID";
```

```
const char* password = "REPLACE_WITH_YOUR_PASSWORD";


// Initialize Telegram BOT

#define BOTtoken "XXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" //
your Bot Token (Get from Botfather)


// Use @myidbot to find out the chat ID of an individual or a group

// Also note that you need to click "start" on a bot before it can
message you

#define CHAT_ID "XXXXXXXXXX"


X509List cert(TELEGRAM_CERTIFICATE_ROOT);

WiFiClientSecure client;

UniversalTelegramBot bot(BOTtoken, client);


const int motionSensor = 14; // PIR Motion Sensor

bool motionDetected = false;


// Indicates when motion is detected

void ICACHE_RAM_ATTR detectsMovement() {

  // Serial.println("MOTION DETECTED!!!");

  motionDetected = true;

}


void setup()

  { Serial.begin(115200);

  configTime(0, 0, "pool.ntp.org"); // get UTC time via NTP

  client.setTrustAnchors(&cert); // Add root certificate for
api.telegram.org


  // PIR Motion Sensor mode INPUT_PULLUP
```

```
  pinMode(motionSensor, INPUT_PULLUP);

  // Set motionSensor pin as interrupt, assign interrupt function and
set RISING mode

  attachInterrupt(digitalPinToInterrupt(motionSensor), detectsMovement,
RISING);



  // Attempt to connect to Wifi network:

  Serial.print("Connecting Wifi: ");

  Serial.println(ssid);



  WiFi.mode(WIFI_STA);

  WiFi.begin(ssid, password);



  while (WiFi.status() != WL_CONNECTED)

    { Serial.print(".");

    delay(500);

  }

  Serial.println("");

  Serial.println("WiFi connected");

  Serial.print("IP address: ");

  Serial.println(WiFi.localIP());



  bot.sendMessage(CHAT_ID, "Bot started up", "");

}

void loop() {

  if (motionDetected) {

    bot.sendMessage(CHAT_ID, "Motion detected!!", "");

    Serial.println("Motion Detected");

    motionDetected = false;

  }

}
```

F. Demonstration/Prosedur

PENTING!: buka akun Telegram Anda dan cari bot Anda. Anda perlu mengklik "/start" pada bot agar dapat mengirimi Anda pesan.

Upload program ke nodeMCU Anda. Jangan lupa untuk pergi ke Tools> Board untuk memilih board. Pergi ke Tools> Port untuk memilih COM yang terhubung ke nodeMCUAnda.

Setelah mengupload program, tekan tombol RST on-board ESP8266/ESP32 untuk memulai proses running program. Kemudian Anda dapat membuka Monitor Serial untuk melihat hasil program yang sedang berjalan. Saat board Anda pertama kali boot, itu akan mengirim pesan ke akun Telegram Anda: "Bot started up". Kemudian, gerakkan tangan Anda di depan sensor gerak PIR dan periksa Telegram apakah Anda telah menerima notifikasi deteksi gerakan.

Sementara itu di serial monitor akan tampil pesan berikut :

G. Hasil Percobaan

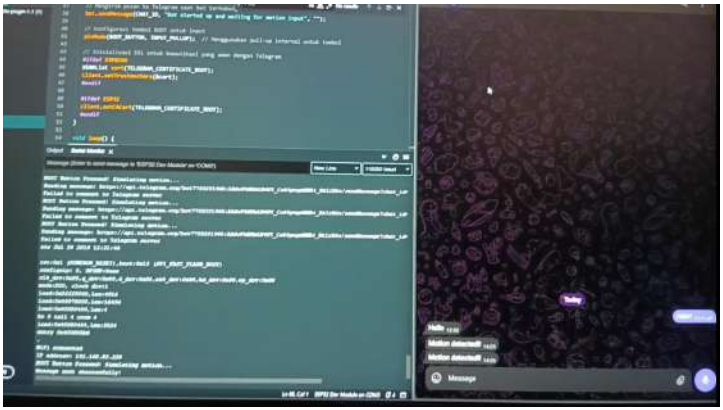
1. Program

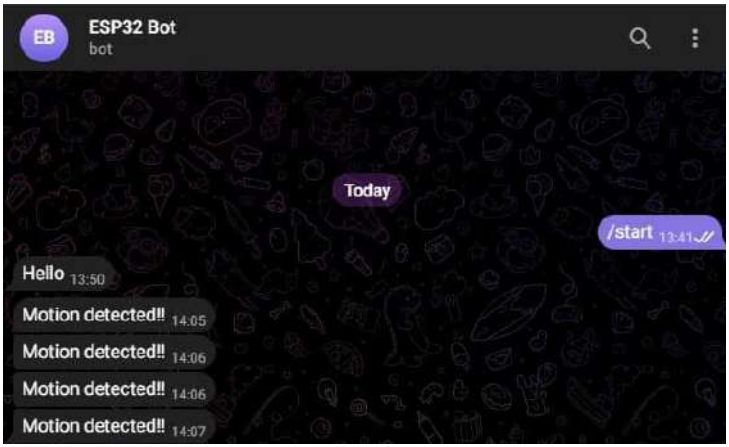
```
#include <Arduino.h>
#ifdef ESP32
#include <WiFi.h>
#else
#include <ESP8266WiFi.h>
#endif
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>

const char *ssid = "miawmiaw";
// Nama SSID WiFi
const char *password = "miawmiaw";
// Password WiFi
#define BOTtoken
"XXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXX"
#define CHAT_ID "XXXXXXXXXX"

WiFiClientSecure client;  // Klien untuk
komunikasi HTTPS dengan Telegram
UniversalTelegramBot bot(BOTtoken, client);
// Membuat objek bot

const int BOOT_BUTTON = 0;  // GPIO 0 (BOOT
button pada ESP32)
bool motionDetected = false;  // Flag untuk
mendeteksi gerakan

void setup() {
  Serial.begin(115200);

  // Menghubungkan ke WiFi
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  Serial.println("\nWiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  // Mengirim pesan ke Telegram saat bot
terhubung
  bot.sendMessage(CHAT_ID, "Bot started up
and waiting for motion input", "");

  // Konfigurasi tombol BOOT untuk input
  pinMode(BOOT_BUTTON, INPUT_PULLUP);  //
Menggunakan pull-up internal untuk tombol

  // Inisialisasi SSL untuk komunikasi yang
aman dengan Telegram
  #ifdef ESP8266
  X509List cert(TELEGRAM_CERTIFICATE_ROOT);
  client.setTrustAnchors(&cert);
  #endif

  #ifdef ESP32

client.setCACert(TELEGRAM_CERTIFICATE_ROOT)
;
  #endif
}

void loop() {
  // Mengecek apakah tombol BOOT ditekan
  if (digitalRead(BOOT_BUTTON) == LOW)
{  // Jika tombol BOOT ditekan
    Serial.println("BOOT Button Pressed!
Simulating motion...");
```

```
    // Kirim pesan ke Telegram
    if (bot.sendMessage(CHAT_ID, "Motion
detected!!", "")) {
      Serial.println("Message sent
successfully!");
    } else {
      Serial.println("Failed to send
message.");
    }
    delay(1000);  // Delay 1 detik untuk
mencegah spam pengiriman pesan
  }
}
```

Hasil :




EB
ESP32 Bot
bot
Today
/start 13:41
Hello 13:50
Motion detected!! 14:05
Motion detected!! 14:06
Motion detected!! 14:06
Motion detected!! 14:07

H. Analisa

Program ini dirancang untuk berjalan pada mikrokontroler **ESP8266** atau **ESP32**, dengan tujuan mengirimkan notifikasi ke Telegram saat tombol **BOOT** ditekan, yang dalam simulasi ini dianggap sebagai deteksi gerakan. Program diawali dengan inisialisasi koneksi WiFi menggunakan **SSID** dan **password** yang telah ditentukan. Jika koneksi berhasil, alamat IP perangkat akan ditampilkan di Serial Monitor sebagai tanda bahwa perangkat telah terhubung ke jaringan. Setelah koneksi terjalin, bot Telegram akan mengirimkan pesan awal ke pengguna sebagai indikasi bahwa sistem telah siap menerima input.

Komunikasi dengan Telegram dilakukan menggunakan **WiFiClientSecure**, yang memungkinkan ESP berkomunikasi dengan server Telegram melalui koneksi HTTPS yang aman. Untuk itu, program mengatur sertifikat SSL root **(TELEGRAM_CERTIFICATE_ROOT)** guna memastikan koneksi terenkripsi dan lebih aman dari serangan man-in-the-middle. Bot Telegram diinisialisasi dengan **BOTtoken** dan **CHAT_ID**, di mana BOTtoken bertindak sebagai kredensial untuk autentikasi ke API Telegram, sementara CHAT_ID menentukan pengguna atau grup yang akan menerima notifikasi.

Tombol BOOT (yang terhubung ke GPIO 0) dikonfigurasi sebagai **INPUT_PULLUP**, yang berarti dalam kondisi default, pin akan berada dalam keadaan HIGH. Ketika tombol ditekan, pin akan membaca nilai LOW, yang akan memicu pengiriman pesan **"Motion detected!!"** ke Telegram. Setelah pengiriman pesan, terdapat **delay selama 1 detik** untuk mencegah pengiriman spam jika tombol ditekan terus-menerus. Namun, mekanisme ini masih bisa ditingkatkan dengan menambahkan **debounce** atau sistem time-based threshold untuk memastikan tombol tidak mengirim banyak pesan secara tidak terkendali dalam waktu singkat.

Secara fungsional, program ini cukup efektif untuk mendeteksi tekanan tombol dan mengirimkan pesan peringatan melalui Telegram. Program juga relatif sederhana dan mudah

dipahami, sehingga cocok digunakan sebagai dasar untuk pengembangan sistem notifikasi berbasis IoT. Namun, ada beberapa **kelemahan** yang perlu diperhatikan. Pertama, tidak adanya mekanisme **retry** jika pengiriman pesan gagal, sehingga jika terjadi gangguan jaringan atau API Telegram tidak merespons, pesan bisa saja hilang tanpa ada upaya pengiriman ulang. Kedua, tidak ada **logika debounce** pada tombol, sehingga jika tombol ditekan berulang kali atau mengalami bouncing akibat gangguan mekanis, ada kemungkinan pesan terkirim berkali-kali dalam waktu singkat. Selain itu, penyimpanan **BOTtoken** dan **CHAT_ID** langsung di dalam kode bisa menjadi risiko keamanan, karena informasi ini bisa dengan mudah terekspos jika kode bocor atau diunggah ke repositori publik seperti GitHub tanpa perlindungan yang memadai.

Dari sisi keamanan, program sudah menggunakan **WiFiClientSecure** dengan sertifikat SSL untuk mengenkripsi komunikasi dengan Telegram, yang merupakan langkah baik untuk melindungi data dari serangan pihak ketiga. Namun, jika perangkat memiliki keterbatasan memori, pengelolaan sertifikat SSL ini bisa menjadi beban tambahan bagi ESP8266. Untuk memperbaiki kelemahan ini, penggunaan metode penyimpanan token yang lebih aman, seperti **EEPROM** atau **SPIFFS**, bisa menjadi solusi.

I. Kesimpulan

Program ini merupakan implementasi sederhana namun efektif dalam menghubungkan ESP8266/ESP32 dengan Telegram untuk mengirimkan notifikasi berdasarkan input dari tombol BOOT. Dengan sedikit modifikasi, program ini bisa dikembangkan menjadi sistem yang lebih kompleks, seperti deteksi gerakan dengan sensor PIR, notifikasi saat ada akses tak sah ke suatu area, atau bahkan integrasi dengan sistem keamanan rumah pintar. Meskipun fungsional, program ini masih bisa ditingkatkan dari segi keamanan, keandalan pengiriman pesan, serta pengelolaan input tombol agar lebih stabil dan tidak mudah terkena gangguan mekanis. Sebagai langkah perbaikan, disarankan untuk menambahkan debounce pada tombol, mengimplementasikan retry mechanism untuk pengiriman pesan, serta menggunakan metode penyimpanan kredensial yang lebih aman. Dengan perbaikan ini, sistem akan menjadi lebih andal dan dapat digunakan dalam berbagai aplikasi IoT berbasis notifikasi real-time.